

ADT

Table of contents

First-Class ADT Pattern	2
First-Class ADT Pattern	2
Encapsulation	2
Abstract Data Type (ADT)	2
ADT in C e in Python	3
First-Class ADT	3
ADT: teoria vs C (con e senza incapsulamento) vs Python	4
STACK	5
Operazioni essenziali	6
Operazioni complementari	6
Operazioni di gestione (tipiche in C)	7
Implementazione dello Stack con Array Statico	7
Scelte progettuali	8
Responsabilità del <code>main</code>	8
Dimensione massima dello stack	8
Assenza di incapsulamento completo	9
Uso dei puntatori	9
Altre note	9
Main minimale	9
<code>module.h</code>	10
<code>module.c</code>	12
Discussione e miglioramenti	15
Exercises	17
Stack using Linked List	20
Stack using a Linked List- only abstraction	23
Stack using a Linked List - First Class ADT	23
QUEUE - (First In First Out policy)	25
put (enqueue) and get (dequeue)	27
DYNAMIC QUEUE (LINKED LIST) (1)	29
DYNAMIC QUEUE (LINKED LIST) (2)	33
QUEUE (using an array)	33

A possible solution	34
Exercises	38

FONTI PRINCIPALI	39
-------------------------	-----------

First-Class ADT Pattern

First-Class ADT Pattern

Encapsulation

L'**incapsulamento** (**encapsulation**) è uno strumento fondamentale per gestire la complessità e separare le responsabilità all'interno di un sistema.

Quando si progetta software, è utile distinguere tra due prospettive:

- **Comportamento esterno (interfaccia):** come un componente viene utilizzato (come interagisce con l'esterno)
- **Implementazione interna:** come il componente è effettivamente costruito (come è strutturato internamente)

Abstract Data Type (ADT)

Un **Tipo di Dato Astratto (ADT)** definisce:

- un **insieme di valori**
- un **insieme di operazioni** su tali valori

senza specificare come questi valori siano rappresentati internamente.

https://en.wikipedia.org/wiki/Abstract_data_type

Vedi anche [Code Complete / Steve McConnell.–2nd ed.]. Un estratto del Cap. 6 è disponibile insieme alle slides.¹

¹“Il riassunto, la citazione o la riproduzione di brani o di parti di opera e la loro comunicazione al pubblico sono liberi se effettuati per uso di critica o di discussione, nei limiti giustificati da tali fini e purché non costituiscano concorrenza all'utilizzazione economica dell'opera; se effettuati a fini di insegnamento o di ricerca scientifica l'utilizzo deve inoltre avvenire per finalità illustrative e per fini non commerciali.” - <https://www.normattiva.it/uri-res/N2Ls?urn:nir:stato:legge:1941-04-22;633~art70>

ADT in C e in Python

Nel linguaggio C, l'incapsulamento di un ADT si realizza tipicamente tramite:

- la dichiarazione di un **tipo incompleto** (*incomplete type*, dato opaco) nel file header `.h`
- la manipolazione dei valori esclusivamente tramite **puntatori**
- la definizione completa della struttura nascosta nel file `.c`

Il tipo è detto *incompleto* perché la sua struttura interna non è visibile al codice che utilizza l'ADT.

In **Python**, gli ADT si implementano tipicamente tramite **classi**, che nascondono (parzialmente) i dettagli interni e forniscono un'interfaccia pubblica.

First-Class ADT

Un ADT è detto **first-class** se i suoi valori possono essere trattati come qualsiasi altro valore del linguaggio.

https://en.wikipedia.org/wiki/First-class_citizen

In C, questo significa che (tipicamente tramite puntatori) un ADT può:

- essere **passato come argomento a una funzione**
- essere **restituito da una funzione**
- essere **assegnato a una variabile**
- essere **memorizzato in strutture dati**

In C, gli ADT sono generalmente gestiti tramite puntatori: l'assegnazione tra variabili copia il puntatore (riferimento), non il contenuto dell'oggetto.

Nel contesto della programmazione, si parla di “**cittadini di prima classe**” (**first-class citizens**) per indicare entità che possono essere usate liberamente: assegnate a variabili, passate come argomenti, restituite da funzioni e memorizzate in strutture dati. In **C**, sono first-class i valori “semplici” (int, float, puntatori, ecc.), mentre le **funzioni non lo sono pienamente**: non possono essere assegnate direttamente a variabili, ma solo tramite **puntatori a funzione**. In **Python**, invece, anche le funzioni sono veri cittadini di prima classe.

```
// Esempio in C
int add(int a, int b) { return a + b; }

int main() {
    int (*f)(int, int) = add; // puntatore a funzione
    int r = f(2, 3);          // uso indiretto
}
```

```
def f1(x):
    print(x)
    #return None
def f2(x):
    print(x)

a = f2 # assegnazione diretta

f1(a) # <function f2 at 0x110211ee0>
```

-
- **ADT con incapsulamento completo dei dati.**
L'attenzione è su **astrazione** e **incapsulamento**: la rappresentazione interna è nascosta e accessibile solo tramite le operazioni definite.
(È un'implementazione **corretta e robusta** di un ADT).
 - **ADT senza incapsulamento completo dei dati.**
L'attenzione è solo sull'**astrazione**: la struttura interna è visibile o direttamente accessibile.
(È un'implementazione **debole** di un ADT, che viola l'incapsulamento).
 - Un ADT può inoltre essere *first-class* se i suoi valori possono essere passati, restituiti e assegnati come qualsiasi altro valore del linguaggio.

ADT: teoria vs C (con e senza incapsulamento) vs Python

Aspetto	Teoria (ADT)	C (con incapsulamento)	C (senza incapsulamento)	Python
Definizione	Insieme di valori + operazioni	Uguale alla teoria	Uguale alla teoria	Uguale alla teoria
Rappresentazione interna	Non specificata	Nascosta (struct in .c)	Visibile (struct in .h)	Attributi della classe

Aspetto	Teoria (ADT)	C (con incapsulamento)	C (senza incapsulamento)	Python
Incapsulamento	Non richiesto	Forte (tipo incompleto)	Assente	Debole (convenzioni)
Accesso ai dati	Tramite operazioni	Solo tramite funzioni	Diretto + funzioni	Diretto + metodi
Modifica dei dati	Tramite operazioni	Tramite funzioni	Diretto + funzioni	Diretto + metodi
First-class	Dipende dal linguaggio	Tramite puntatori	Tramite puntatori	Nativamente

STACK

[https://en.wikipedia.org/wiki/Stack_\(abstract_data_type\)](https://en.wikipedia.org/wiki/Stack_(abstract_data_type))

Lo **stack** (o **pila**) è un ADT in cui gli elementi vengono gestiti secondo la politica **LIFO** (*Last In, First Out*): l'ultimo elemento inserito è il primo a uscire.

Le operazioni fondamentali sono:

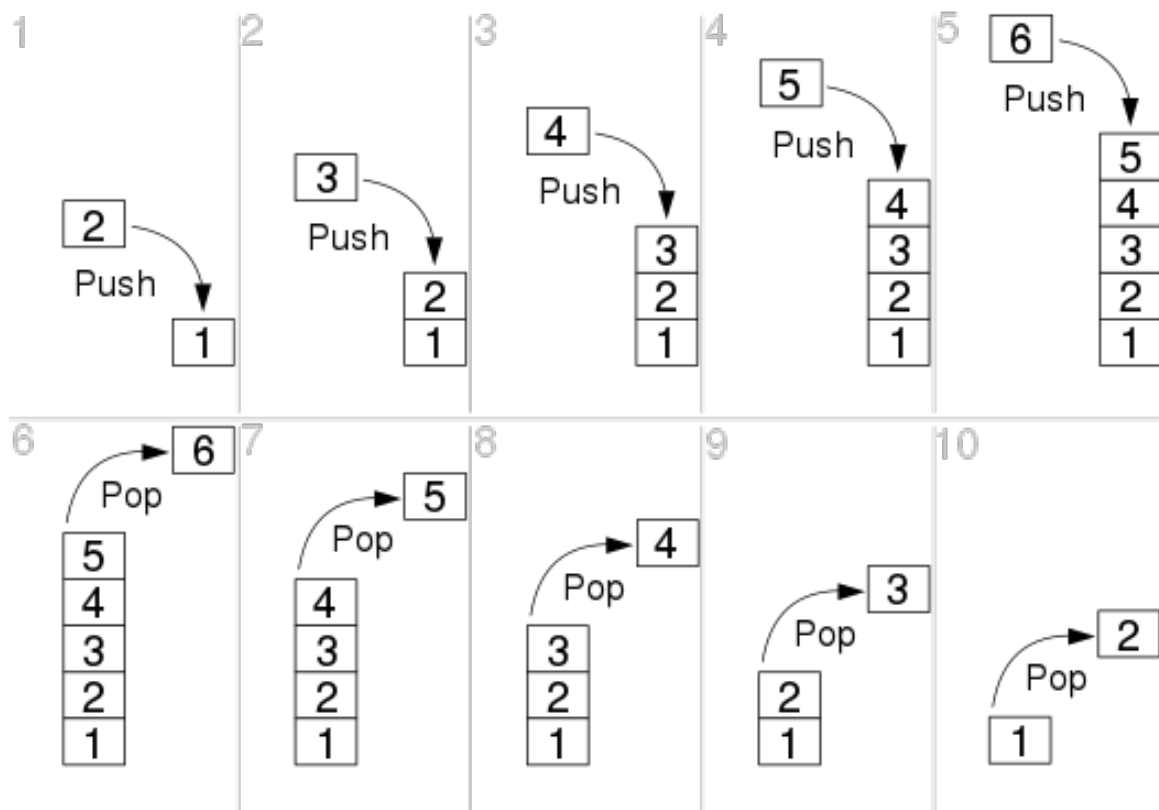
- **push**: inserisce un nuovo elemento in cima allo stack
- **pop**: rimuove l'elemento in cima allo stack
- **peek**: restituisce l'elemento in cima senza rimuoverlo

Tutte queste operazioni agiscono su un'unica estremità della struttura, detta **cima** (*top*).

In C, lo stack può essere implementato con:

- un **array**
- una **lista concatenata**

La scelta dell'implementazione influenza gli aspetti pratici (uso della memoria, dimensione massima, efficienza), ma non modifica il comportamento astratto dello stack. Uno stack è definito dalle sue operazioni e dal suo comportamento LIFO, non dal modo in cui viene implementato.



Operazioni essenziali

Sono le operazioni **minime** che definiscono il comportamento dello stack:

- `push(Stack* s, T x)`
Inserisce un elemento in cima allo stack
- `pop(Stack* s)`
Rimuove l'elemento in cima allo stack

Operazioni complementari

Non sono strettamente necessarie per definire l'ADT, ma sono **molto utili** nella pratica:

- `peek(const Stack* s) / peek(...)`
Restituisce l'elemento in cima senza rimuoverlo
- `isEmpty(const Stack* s)`
Verifica se lo stack è vuoto

- `size(const Stack* s)`
Restituisce il numero di elementi

Operazioni di gestione (tipiche in C)

In C sono fondamentali per gestire memoria e ciclo di vita:

- `create() / init()`
Crea e inizializza uno stack
 - `destroy(Stack* s)`
Libera la memoria associata
 - `isFull(const Stack* s)` ‘non è un’operazione propria dello stack come ADT, ma può essere introdotta in implementazioni con capacità limitata.
-

Implementazione dello Stack con Array Statico

Uno stack può essere implementato utilizzando:

- un **array statico** per memorizzare gli elementi
- una variabile intera **top** che indica la posizione dell’elemento in cima allo stack

```
typedef struct stack {  
    int stackArray[N];  
    int top;  
} Stack;
```

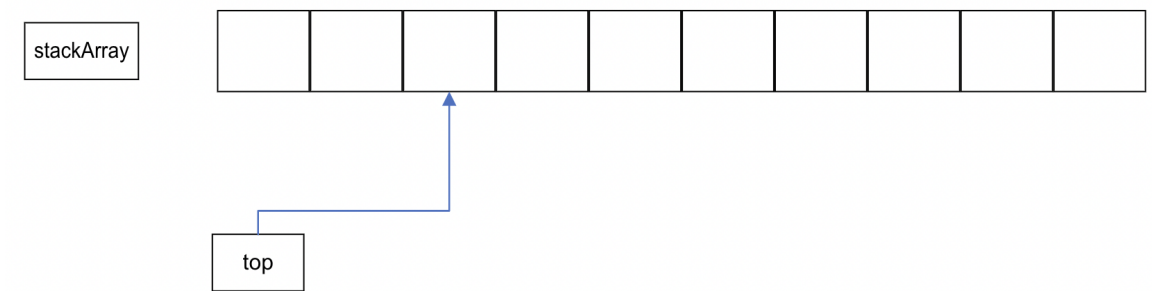
Quando lo stack è vuoto, **top** vale:

```
top = -1;
```

Le operazioni di inserimento o estrazione modificano **top**:

- **push** incrementa **top** e inserisce un elemento
- **pop** restituisce l’elemento in cima e decrementa **top**

In questo modo, **top** rappresenta sempre l’**indice dell’ultimo elemento** inserito nello stack.



Scelte progettuali

Questa versione è **semplificata e intenzionalmente non completa** dal punto di vista progettuale.

Responsabilità del main

In questa implementazione, il controllo delle condizioni di corretto utilizzo dello stack è delegato al programma chiamante (**main**).

In particolare:

- prima di eseguire una **pop** o **peek**, il **main** deve verificare che lo stack **non sia vuoto** mediante la funzione `isEmpty()`
- prima di eseguire una **push**, il **main** deve verificare che lo stack **non sia pieno** mediante la funzione `isFull()`

Questa è una **scelta didattica**:

le funzioni del modulo non effettuano controlli interni e richiedono il rispetto di precise **precondizioni**.

Dimensione massima dello stack

La dimensione dello stack è fissata a compile-time tramite una direttiva:

```
#define STACK_MAX_SIZE 100
```

Questa costante è definita nel file di interfaccia (`.h`) ed è quindi **visibile a tutto il programma**. Si tratta di una scelta che privilegia semplicità e chiarezza del modello a scapito della flessibilità.

Assenza di incapsulamento completo

La struttura è visibile nell'interfaccia `.h`. Il main può bypassare le funzioni di interfaccia.

```
s->stackArray[0]=10;
s->top=0;
```

Uso dei puntatori

Le funzioni operano su **puntatori a Stack**. Questa scelta è motivata da **efficienza**, perché si evita la copia della struttura, e **coerenza progettuale**, perché facilita l'evoluzione verso implementazioni più avanzate, per esempio incapsulamento completo. Tuttavia questo non è un obbligo. Dato che la struttura è visibile nell'interfaccia, il main può dichiarare direttamente variabili `struct stack` e manipolare i campi.

Altre note

Questa implementazione presenta alcune limitazioni intenzionali:

- non verifica condizioni di errore (overflow/underflow)
- espone direttamente la struttura dati
- utilizza una dimensione fissa

Tali scelte la rendono semplice da comprendere e adatta a una prima introduzione, ma spesso non adeguata a contesti reali.

Viene fornito un main con un menu ed un test minimali, e una funzione `show()` che mostra tutti gli elementi dello stack. La funzione `show()` è solo per debug, e dovrebbe essere rimossa dal codice.

Main minimale

```
int main() {
    int value = 0;
    Stack *s = initStack();

    // Push (with check)
    if (!isFull(s)) {
        push(s, value);
    } else {
        printf("Stack pieno!\n");
    }
}
```

```

    // Peek (with check)
    if (!isEmpty(s)) {
        value = peek(s);
    } else {
        printf("Stack vuoto!\n");
    }

    // Pop (with check)
    if (!isEmpty(s)) {
        value = pop(s);
    } else {
        printf("Stack vuoto!\n");
    }

    // ora lo stack è vuoto

    push(s, 1);
    push(s, 2);
    push(s, 3);

    show(s);
}

```

module.h

```

// ADT/stack_array_noencasp/module.h

/* STACK
 * Implementazione: array statico
 * SENZA incapsulamento completo
 *
 * È possibile aggirare l'interfaccia e violare l'incapsulamento:
 * s->stackArray[0] = 10;
 * s->top = 0;
 *
 * L'implementazione utilizza un array contenuto in una struttura.
 * L'allocazione della struttura è dinamica, ma questo è solo un dettaglio implementativo.
 * Modificando initStack si potrebbe ottenere un'implementazione completamente statica.
 */

```

```

#ifndef MODULE_H
#define MODULE_H
#include <stdbool.h>
#include <stdlib.h>
#include <stdio.h>

/* STACK_MAX_SIZE -> capacity of the Static Stack
 * STACK_MAX_SIZE is part of the interface
 * and it is visible from main()
 */
#define STACK_MAX_SIZE 1000

typedef struct stack Stack;

struct stack {
    int stackArray[STACK_MAX_SIZE];
    int top;
};

// Function prototypes

// Push element to the top of the stack
// PRECONDIZIONE: stack NON pieno
void push(Stack * stack, int val);

// Remove and return the top most element of the stack
// PRECONDIZIONE: stack NON vuoto
int pop(Stack * stack);

// Return the top most element of the stack
// PRECONDIZIONE: stack NON vuoto
int peek(Stack * stack);

// Check if the stack is in Underflow state or not
bool isEmpty(Stack * stack);

Stack * initStack(); //INIT
void deleteStack(Stack * stack); // Free memory

// Check if the stack is in Overflow state or not
// ONLY FOR STATIC ARRAY IMPLEMENTATION
bool isFull(Stack * stack);

// FOR DEBUG USE ONLY - REMOVE FROM THE INTERFACE

```

```

void show(Stack * stack);
void automaticTest(void);

#endif //MODULE_H

```

module.c

```

// ADT/stack_array_noencasp/module.c

#include <assert.h>
#include "module.h"

Stack * initStack(){
    Stack * stack= malloc(sizeof(*stack));
    if (!stack) {
        printf("Memory allocation failed!\n");
        exit(EXIT_FAILURE); // Stop the program if malloc fails
    }
    stack->top=-1;
    return stack;
};

void deleteStack(Stack * p_s){
    // attenzione, dangling pointer
    // il chiamante mantiene un puntatore dangling se non lo azzerava con p=NULL
    free (p_s);
}

void push(Stack * stack, int val){

    stack->top+=1;
    stack->stackArray[stack->top] = val;
}

// PRECONDIZIONE: stack NON vuoto
int pop(Stack * stack){
    int val = stack->stackArray[stack->top];
    stack->top-=1;
    return val;
}

```

```

// PRECONDIZIONE: stack NON vuoto
int peek(Stack * stack){
    int val = stack->stackArray[stack->top];
    return val;
}

bool isEmpty(Stack * stack){ return stack->top==-1; }

bool isFull(Stack * stack){ return stack->top == STACK_MAX_SIZE-1; }

// FOR DEBUG USE ONLY - REMOVE FROM THE INTERFACE
void show(Stack * stack){
    int i;
    if (isEmpty( stack)) {
        printf("Empty stack!\n");
        return;
    }
    printf("[TOP] ");
    for (i=stack->top; i>=0; i--)
        printf("%d ",stack->stackArray[i]);
    printf("] [BASE]\n");
}

void automaticTest(void) {
    int value;
    Stack * s;
    s = initStack();
    printf("Automatic test\n");
    printf(" - check empty\n");
    assert(isEmpty(s)); //https://en.wikipedia.org/wiki/Assert.h
    printf(" - check push, pop, peek\n");
    // Push 1
    value=1;
    push(s, value);
    assert( !(isEmpty(s)));
    assert(value == peek(s));
    assert( !(isEmpty(s)));
    assert(value == pop(s));
    assert((isEmpty(s)));

    // Push 1, 2, 3
    push(s, 1);

```

```

push(s, 2);
push(s, 3);
printf("->expected: \n[TOP] [3 2 1] [BASE]\n");
printf("->obtained:\n");
show(s);
assert( !(isEmpty(s)) );
assert(3 == peek(s));
assert(3 == pop(s));
assert(2 == peek(s));
assert(2 == pop(s));
assert(1 == peek(s));
assert(1 == pop(s));
assert((isEmpty(s)));

printf("OK!\n");
printf("-----\n");
// Don't forget to free the memory!
deleteStack(s);
}

```